- 🗺️ How to Use — Hướng Dẫn Sử Dụng Antigravity Kit
  - Nguyên Tắc Vàng
  - Mục Lục
  - A. Web App
    - Combo cần dùng
    - ❌ KHÔNG cần
    - Chu trình thực tế
    - Ví dụ prompt thực tế
  - B. Landing Page / Marketing Site
    - Combo cần dùng
    - ❌ KHÔNG cần
    - Chu trình thực tế
    - Ví dụ prompt thực tế
  - C. REST API / Backend Service
    - Combo cần dùng
    - ❌ KHÔNG cần
    - Chu trình thực tế
    - Ví dụ prompt thực tế
  - D. Mobile App
    - Combo cần dùng
    - ❌ KHÔNG cần
    - ⚠️ QUY TẮC QUAN TRỌNG
    - Chu trình thực tế
    - Ví dụ prompt thực tế
  - E. Game Development
    - Combo cần dùng
    - ❌ KHÔNG cần
    - Chu trình thực tế
    - Ví dụ prompt thực tế
  - F. E-commerce / SEO-heavy Site
    - Combo cần dùng
    - Đây là loại project CẦN NHIỀU NHẤT
    - Chu trình thực tế
  - 📋 Bảng Tra Nhanh: Project → Workflow
  - 🔧 7 Workflow Hỗ Trợ — Dùng Khi Nào?
    - 1. /brainstorm — Khi ý tưởng còn mơ hồ
    - 2. /plan — Khi cần chia nhỏ công việc
    - 3. /enhance — Khi thêm tính năng vào project SẴN CÓ
    - 4. /preview — Khi cần xem kết quả trên browser
    - 5. /status — Khi cần biết mình đang ở đâu
    - 6. /code-simplify — Khi code "phình cồng kềnh"
    - 7. /orchestrate — Khi cần HỘI ĐỒNG nhiều agents
  - 📅 Vòng Đời Hoàn Chỉnh Của 1 Project
  - 🧠 Mẹo Sử Dụng

- 1. Bắt đầu luôn bằng /spec
- 2. Đừng skip /review
- 3. Dùng /code-simplify khi code bắt đầu "nặng"
- 4. /status là "bản đồ" của bạn
- 5. /orchestrate = vũ khí hạng nặng
- 6. Skill tự load — bạn KHÔNG cần gọi
- 7. Tag agent khi muốn chính xác
- 8. Khi bị lỗi → /debug trước, KHÔNG tự sửa
- 9. Combo hay dùng nhất hàng ngày
- 10. Combo làm sạch code

🗺️ How to Use — Hướng Dẫn Sử Dụng Antigravity Kit

> Hướng dẫn thực chiến: Dùng combo **Workflows + Skills + Agents** nào cho từng loại dự án. Không phải dự án nào cũng cần tất cả. Chọn đúng combo, tiết kiệm token, tăng hiệu quả.

Nguyên Tắc Vàng

```
1. Workflow  = Bạn gõ lệnh (ví dụ /spec, /plan)
2. Agent     = AI tự chọn chuyên gia phù hợp (hoặc bạn tag @agent)
3. Skill     = Kiến thức AI tự load khi cần (bạn KHÔNG cần gọi)
```

Bạn chỉ cần nhớ Workflows. Agent và Skill tự hoạt động.

Mục Lục

#	Loại Project	Đi tới
A	Web App (Todolist, Dashboard, CRM...)	Xem combo A
B	Landing Page / Marketing Site	Xem combo B
C	REST API / Backend Service	Xem combo C
D	Mobile App (React Native / Flutter)	Xem combo D
E	Game (2D/3D Browser Game)	Xem combo E
F	E-commerce / SEO-heavy Site	Xem combo F

A. Web App

> **Ví dụ:** Todolist, Dashboard quản lý, CRM, Chat app, Kanban board

Combo cần dùng

Loại	Tên	Vai trò
🔄 Workflow	`/spec` → `/plan` → `/create` → `/review` → `/test`	Quy trình chính
🤖 Agent	`frontend-specialist`, `backend-specialist`	AI tự chọn
📚 Skill	`clean-code`, `react-best-practices`, `testing-patterns`, `source-driven-development`	AI tự load

❌ KHÔNG cần

`game-development`, `mobile-design`, `seo-fundamentals`, `geo-fundamentals`, `red-team-tactics`, `rust-pro`

Chu trình thực tế

```
Bước 1: /spec todolist app with drag-and-drop
        → AI hỏi bạn 3-5 câu → Tạo file SPEC.md
        → Skills tự load: brainstorming

Bước 2: /plan
        → AI đọc SPEC.md → Chia thành 5-8 tasks nhỏ
        → Skills tự load: plan-writing

Bước 3: /create
        → AI bắt đầu code theo plan
        → Agents tự chọn: frontend-specialist (UI) + backend-specialist (API)
        → Skills tự load: clean-code, react-best-practices, source-driven-development

Bước 4: /preview
        → Bật server xem kết quả

Bước 5: /review
        → AI review code 5 trục (Correctness, Security, Performance...)
        → Skills tự load: code-review-checklist, vulnerability-scanner

Bước 6: /test
        → Tạo + chạy test
        → Skills tự load: testing-patterns, tdd-workflow

Bước 7 (nếu code rối): /code-simplify src/components/
        → Dọn dẹp code thừa
        → Skills tự load: clean-code
```

Ví dụ prompt thực tế

```
Bạn: /spec todolist app với drag and drop, lưu localStorage, dark mode
Bạn: /plan
Bạn: /create
Bạn: /preview
Bạn: /review
Bạn: Code hơi rối, /code-simplify src/components/TaskList.tsx
Bạn: /test src/services/
```

B. Landing Page / Marketing Site

> **Ví dụ:** Portfolio, startup landing page, product showcase, blog

Combo cần dùng

Loại	Tên	Vai trò
🔄 Workflow	`/spec` → `/plan` → `/create` → `/ui-ux-pro-max` → `/review`	Quy trình chính
🤖 Agent	`frontend-specialist`, `seo-specialist`	AI tự chọn
📚 Skill	`frontend-design`, `seo-fundamentals`, `geo-fundamentals`, `clean-code`	AI tự load

❌ KHÔNG cần

`backend-specialist`, `database-design`, `api-patterns`, `game-development`, `mobile-design`, `red-team-tactics`

Chu trình thực tế

```
Bước 1: /spec landing page for AI SaaS product
        → Chốt sections, target audience, CTA goals

Bước 2: /plan
        → Chia: Hero → Features → Pricing → Footer

Bước 3: /create
        → Agent: frontend-specialist
        → Skills: clean-code, source-driven-development

Bước 4: /ui-ux-pro-max
        → ĐÂY LÀ BƯỚC QUAN TRỌNG NHẤT cho landing page
        → Agent: frontend-specialist + frontend-design skill
        → Gradient, animation, typography premium
```

```
Bước 5: /review
        → Kiểm tra: Accessibility, Performance, SEO
        → Skills tự load: seo-fundamentals, geo-fundamentals

Bước 6: /deploy
        → Pre-launch checklist + SEO verification
```

Ví dụ prompt thực tế

```
Bạn: /spec landing page cho app AI viết CV, target GenZ
Bạn: /plan
Bạn: /create
Bạn: /ui-ux-pro-max modern, glassmorphism, dark theme
Bạn: /review all
Bạn: /deploy
```

C. REST API / Backend Service

> **Ví dụ:** API cho mobile app, Microservice, Webhook server, Auth service

Combo cần dùng

Loại	Tên	Vai trò
🔄 Workflow	`/spec` → `/plan` → `/create` → `/test` → `/review` → `/deploy`	Quy trình chính
🤖 Agent	`backend-specialist`, `security-auditor`, `database-architect`	AI tự chọn
📚 Skill	`api-patterns`, `database-design`, `nodejs-best-practices`, `testing-patterns`, `vulnerability-scanner`	AI tự load

❌ KHÔNG cần

`frontend-design`, `mobile-design`, `seo-fundamentals`, `geo-fundamentals`, `game-development`, `tailwind-patterns`

Chu trình thực tế

```
Bước 1: /spec REST API for user management with JWT auth
        → Chốt: endpoints, auth method, database, rate limiting

Bước 2: /plan
        → Chia: DB schema → Auth middleware → CRUD routes → Validation
```

```
Bước 3: /create
        → Agent: backend-specialist (API) + database-architect (schema)
        → Skills: api-patterns, database-design, nodejs-best-practices

Bước 4: /test
        → Unit tests cho mỗi route + integration test
        → Skills: testing-patterns, tdd-workflow

Bước 5: /review
        → ĐẶC BIỆT QUAN TRỌNG cho backend: Axis Security + Performance
        → Agent: security-auditor tự activate
        → Skills: vulnerability-scanner, red-team-tactics

Bước 6: /deploy
        → Pre-launch: security scan, env vars, migration ready
        → Skills: deployment-procedures, server-management
```

Ví dụ prompt thực tế

```
Bạn: /spec REST API quản lý user, JWT auth, PostgreSQL, rate limiting
Bạn: /plan
Bạn: /create
Bạn: /test src/routes/
Bạn: /review (focus security)
Bạn: /deploy
```

D. Mobile App

> **Ví dụ:** Fitness tracker, Social media app, E-wallet, Food delivery

Combo cần dùng

Loại	Tên	Vai trò
🔄 Workflow	`/spec` → `/plan` → `/create` → `/review` → `/test`	Quy trình chính
🤖 Agent	`mobile-developer` (KHÔNG DÙNG frontend-specialist!)	AI tự chọn
📚 Skill	`mobile-design`, `clean-code`, `testing-patterns`, `api-patterns`	AI tự load

❌ KHÔNG cần

`frontend-design`, `seo-fundamentals`, `geo-fundamentals`, `tailwind-patterns`, `game-development`, `webapp-testing`

⚠️ QUY TẮC QUAN TRỌNG

```
🔴 Mobile + frontend-specialist = SAI
✅ Mobile = mobile-developer ONLY
```

Chu trình thực tế

```
Bước 1: /spec mobile fitness tracker, React Native, track workouts
        → Chốt: platform (iOS/Android/cả hai), navigation, data storage

Bước 2: /plan
        → Chia: Navigation → Home screen → Workout logger → History → Settings

Bước 3: /create
        → Agent: mobile-developer (KHÔNG PHẢI frontend-specialist)
        → Skills: mobile-design, clean-code

Bước 4: /review
        → Focus: Touch targets, gesture handling, offline support
        → Skills: mobile-design (audit), performance-profiling

Bước 5: /test
        → Component tests + integration tests
        → Skills: testing-patterns
```

Ví dụ prompt thực tế

```
Bạn: /spec React Native fitness app, track sets/reps, offline-first
Bạn: /plan
Bạn: @mobile-developer /create
Bạn: /review (focus mobile UX)
Bạn: /test
```

E. Game Development

> **Ví dụ:** 2D platformer, Puzzle game, Card game, Browser RPG

Combo cần dùng

Loại	Tên	Vai trò
🔄 Workflow	`/spec` → `/plan` → `/create` → `/review` → `/test`	Quy trình chính
🤖 Agent	`game-developer`	AI tự chọn
📚 Skill	`game-development`, `clean-code`, `performance-profiling`	AI tự load

❌ KHÔNG cần

`seo-fundamentals`, `geo-fundamentals`, `api-patterns`, `database-design`, `mobile-design`, `frontend-design`, `tailwind-patterns`, `deployment-procedures`, `vulnerability-scanner`

Chu trình thực tế

```
Bước 1: /spec 2D platformer game, HTML5 Canvas, keyboard controls
        → Chốt: engine (Phaser/PixiJS/vanilla Canvas), mechanics, levels

Bước 2: /plan
        → Chia: Game loop → Player → Physics → Enemies → Levels → UI

Bước 3: @game-developer /create
        → Agent: game-developer
        → Skills: game-development, clean-code

Bước 4: /preview
        → Chơi thử trên browser

Bước 5: /review
        → Focus: Performance (60fps), memory leaks, game balance
        → Skills: performance-profiling

Bước 6: /test
        → Unit test cho game logic (physics, scoring, collision)
        → KHÔNG test rendering — test LOGIC
```

Ví dụ prompt thực tế

```
Bạn: /spec browser puzzle game, match-3 mechanics, Phaser.js
Bạn: /plan
Bạn: @game-developer /create
Bạn: /preview
Bạn: Game bị lag, /review (focus performance)
Bạn: /test src/game/logic/
```

F. E-commerce / SEO-heavy Site

> **Ví dụ:** Online store, Blog platform, News site, Marketplace

Combo cần dùng

Loại	Tên	Vai trò
🔄 Workflow	`/spec` → `/plan` → `/create` → `/ui-ux-pro-max` → `/review` → `/test` → `/deploy`	Quy trình đầy đủ nhất
🤖 Agent	`frontend-specialist`, `backend-specialist`, `seo-specialist`, `security-auditor`	Nhiều agents
📚 Skill	`frontend-design`, `seo-fundamentals`, `geo-fundamentals`, `api-patterns`, `database-design`, `vulnerability-scanner`	AI tự load

Đây là loại project CẦN NHIỀU NHẤT

```
SEO         → seo-fundamentals + geo-fundamentals (AI search optimization)
Security    → vulnerability-scanner + red-team-tactics (payment, user data)
Performance → performance-profiling (Core Web Vitals cho SEO ranking)
Database    → database-design + prisma-expert (product catalog, orders)
```

Chu trình thực tế

```
Bước 1: /spec e-commerce site, Next.js, Stripe payment, product catalog
        → Chốt: SEO requirements, payment flow, product schema

Bước 2: /plan
        → Chia: DB schema → Product pages → Cart → Checkout → Payment → SEO

Bước 3: /create
        → Agents: frontend-specialist (UI) + backend-specialist (API)
        → Skills: react-best-practices, api-patterns, database-design

Bước 4: /ui-ux-pro-max
        → Product pages phải đẹp, CTA rõ ràng
        → Skills: frontend-design

Bước 5: /review
        → Full 5-trục review + EXTRA focus Security (payment!) + SEO
        → Agents: security-auditor + seo-specialist
        → Skills: vulnerability-scanner, seo-fundamentals, geo-fundamentals

Bước 6: /test
        → Unit + E2E (đặc biệt checkout flow!)
        → Skills: testing-patterns, webapp-testing

Bước 7: /deploy
        → Pre-launch: security scan + SEO audit + performance check
        → Skills: deployment-procedures, performance-profiling
```

📋 Bảng Tra Nhanh: Project → Workflow

Giai đoạn	Web App	Landing	API	Mobile	Game	E-commerce
Ý tưởng	`/brainstorm`	`/brainstorm`	`/brainstorm`	`/brainstorm`	`/brainstorm`	`/brainstorm`
Chốt spec	`/spec`	`/spec`	`/spec`	`/spec`	`/spec`	`/spec`
Lập kế hoạch	`/plan`	`/plan`	`/plan`	`/plan`	`/plan`	`/plan`
Xây dựng	`/create`	`/create`	`/create`	`/create`	`/create`	`/create`
Thiết kế đẹp	—	`/ui-ux-pro-max`	—	—	—	`/ui-ux-pro-max`
Thêm tính năng	`/enhance`	—	`/enhance`	`/enhance`	`/enhance`	`/enhance`
Xem kết quả	`/preview`	`/preview`	—	—	`/preview`	`/preview`
Tiến độ	`/status`	—	`/status`	`/status`	—	`/status`
Review code	`/review`	`/review`	`/review`	`/review`	`/review`	`/review`
Dọn dẹp code	`/code-simplify`	—	`/code-simplify`	`/code-simplify`	`/code-simplify`	`/code-simplify`
Test	`/test`	—	`/test`	`/test`	`/test`	`/test`
Debug	`/debug`	`/debug`	`/debug`	`/debug`	`/debug`	`/debug`
Multi-agent	`/orchestrate`	—	`/orchestrate`	—	—	`/orchestrate`
Deploy	`/deploy`	`/deploy`	`/deploy`	—	—	`/deploy`

🔧 7 Workflow Hỗ Trợ — Dùng Khi Nào?

Ngoài luồng chính (`/spec` → `/plan` → `/create` → `/review` → `/test` → `/deploy`), có 7 workflow hỗ trợ dùng ở **các thời điểm cụ thể** trong vòng đời dự án:

1. `/brainstorm` — Khi ý tưởng còn mơ hồ

```
📍 Giai đoạn: TRƯỚC /spec (hoặc khi bắt đầu session mới)
🎯 Mục đích:  AI hỏi bạn câu hỏi Socratic để làm rõ yêu cầu
⏱ Khi nào:   Bạn biết muốn gì nhưng chưa biết bắt đầu từ đâu
```

Dùng khi:

- Ý tưởng mới chưa rõ scope
- Muốn so sánh 2-3 hướng tiếp cận trước khi chọn
- Project lớn, cần khám phá trước khi commit spec

KHÔNG dùng khi:

- Bạn đã biết rõ 100% muốn gì → skip thẳng `/spec`
- Sửa bug đơn giản → `/debug`

Ví dụ thực tế:

```
Bạn: /brainstorm tôi muốn làm app quản lý tài chính cá nhân
AI:  Hỏi 3-5 câu: Target ai? Web hay mobile? Features chính?
Bạn: (trả lời)
AI:  Tổng hợp 2-3 hướng → bạn chọn
Bạn: /spec (chuyển sang spec với ý tưởng đã rõ)
```

---

2. `/plan` — Khi cần chia nhỏ công việc

```
📍 Giai đoạn: SAU /spec, TRƯỚC /create
🎯 Mục đích:  Chia dự án thành tasks nhỏ có verify criteria
⏱ Khi nào:   Project có >3 tính năng hoặc >5 files
```

Dùng khi:

- Project trung bình-lớn (5+ files)
- Nhiều tính năng cần build theo thứ tự
- Muốn AI tự track tiến độ

KHÔNG dùng khi:

- Project 1-2 files → `/create` trực tiếp
- Chỉ sửa 1 bug → `/debug`

Ví dụ thực tế:

```
Bạn: /plan
AI:  Đọc SPEC.md → tạo docs/PLAN-todolist.md
     Task 1: Setup project → verify: server chạy
     Task 2: Database schema → verify: migration OK
```

```
    Task 3: CRUD API → verify: Postman test pass
    ...
```

---

3. `/enhance` — Khi thêm tính năng vào project SẴN CÓ

```
📍 Giai đoạn: SAU /create (khi project đã chạy được)
🎯 Mục đích:  Thêm feature mới KHÔNG phá code cũ
⏱ Khi nào:   Project đã có base, muốn mở rộng
```

Dùng khi:

- Thêm dark mode vào app đã có
- Thêm search, filter, pagination
- Thêm admin panel, notification
- Iterate sau demo đầu tiên

KHÔNG dùng khi:

- Project chưa tồn tại → `/create`
- Sửa bug → `/debug`
- Refactor lớn → `/code-simplify` hoặc `/orchestrate`

Ví dụ thực tế:

```
Bạn: /enhance thêm dark mode toggle
Bạn: /enhance thêm search filter cho product list
Bạn: /enhance integrate Stripe payment
Bạn: /enhance thêm đa ngôn ngữ (i18n)
```

⚠️ **Mẹo:** Sau 3-4 lần `/enhance`, chạy `/code-simplify` + `/review` để dọn dẹp.

---

4. `/preview` — Khi cần xem kết quả trên browser

```
📍 Giai đoạn: SAU mỗi lần /create hoặc /enhance
🎯 Mục đích:  Bật dev server, mở browser xem UI
⏱ Khi nào:   Muốn kiểm tra bằng mắt
```

Dùng khi:

- Sau `/create` — xem lần đầu
- Sau `/enhance` — xem tính năng mới
- Sau `/ui-ux-pro-max` — xem thiết kế
- Sau `/code-simplify` — confirm UI không bị vỡ

KHÔNG dùng khi:

- Backend API (không có UI) → `/test` thay thế
- Mobile app → chạy emulator riêng

Ví dụ thực tế:

```
Bạn: /preview           ← bật server
Bạn: /preview status    ← check server đang chạy?
Bạn: /preview stop      ← tắt server
```

---

5. `/status` — Khi cần biết mình đang ở đâu

```
📍 Giai đoạn: BẤT KỲ LÚC NÀO (giữa project)
🎯 Mục đích:  Xem tiến độ, tasks hoàn thành, tasks pending
⏱ Khi nào:   Project lớn, session dài, mất phương hướng
```

Dùng khi:

- Bắt đầu session mới sau khi tắt máy
- Quên mình đã làm đến đâu
- Muốn xem AI đã hoàn thành bao nhiêu tasks trong plan
- Check server có đang chạy không

KHÔNG dùng khi:

- Project nhỏ (1-2 files) → không cần track
- Vừa mới `/create` xong → chưa có gì để track

Ví dụ thực tế:

```
(Sáng hôm sau mở máy lại)
Bạn: /status
AI:  ✅ 5/8 tasks done, ⏳ 3 pending
     Server: not running
     Last change: src/components/Cart.tsx

Bạn: /preview   ← bật lại server
Bạn: /enhance tiếp task 6
```

---

6. `/code-simplify` — Khi code "phình cồng kềnh"

```
📍 Giai đoạn: SAU nhiều lần /enhance hoặc /debug
🎯 Mục đích:  Giảm complexity, giữ nguyên behavior
```

```
⏱ Khi nào:   Code dài, nested, khó đọc
```

Dùng khi:

- Sau 3-4 lần `/enhance` → code bloated
- Function >50 dòng
- Nesting >3 levels
- AI viết quá phức tạp, bạn muốn đơn giản lại
- Trước `/review` để review code sạch hơn

KHÔNG dùng khi:

- Code đang chạy tốt, không rối → đừng động vào
- Muốn thêm feature → `/enhance`
- Muốn fix bug → `/debug`

Ví dụ thực tế:

```
Bạn: /code-simplify src/services/payment.ts
Bạn: /code-simplify recently changed files
Bạn: /code-simplify src/components/Dashboard.tsx
```

⚠️ **Luôn chạy `/test` SAU `/code-simplify`** để đảm bảo không vỡ gì.

---

7. `/orchestrate` — Khi cần HỘI ĐỒNG nhiều agents

```
📍 Giai đoạn: PROJECT LỚN hoặc QUYẾT ĐỊNH PHỨC TẠP
🎯 Mục đích:  Gọi 3+ agents cùng lúc phối hợp
⏱ Khi nào:   Task yêu cầu kiến thức đa lĩnh vực
```

Dùng khi:

- Build feature full-stack (cần cả frontend + backend + database + security)
- Quyết định kiến trúc lớn (chọn tech stack, database design)
- Security audit toàn diện trước deploy
- Refactor lớn ảnh hưởng nhiều module

KHÔNG dùng khi:

- Task đơn giản 1 lĩnh vực → AI tự chọn 1 agent
- Sửa bug → `/debug`
- Thêm 1 feature nhỏ → `/enhance`

Ví dụ thực tế:

```
Bạn: /orchestrate build full authentication system with OAuth, 2FA, and rate limiting

AI sẽ:
  Phase 1: project-planner → tạo PLAN.md
  (Chờ bạn approve)
  Phase 2:
    → database-architect: user + session schema
    → backend-specialist: auth API + middleware
    → security-auditor: review vulnerabilities
    → test-engineer: E2E tests cho auth flow
  Phase 3: Verification scripts
```

⚠️ `/orchestrate` **tốn nhiều token nhất** — chỉ dùng cho task thực sự phức tạp.

📅 Vòng Đời Hoàn Chỉnh Của 1 Project

```
┌──────────────────────────────────────────────────────┐
│                      KHỞI ĐỘNG                       │
│  /brainstorm (nếu mơ hồ) → /spec → /plan             │
└──────────────────────────────────────────────────────┘
                         │
                         ▼
┌──────────────────────────────────────────────────────┐
│                      XÂY DỰNG                        │
│  /create → /preview → kiểm tra bằng mắt              │
└──────────────────────────────────────────────────────┘
                         │
                         ▼
┌──────────────────────────────────────────────────────┐
│                  PHÁT TRIỂN TIẾP                     │
│  /enhance (lặp lại) → /preview → /status             │
│  Nếu code rối: /code-simplify                        │
│  Nếu bug: /debug                                     │
│  Nếu phức tạp: /orchestrate                          │
└──────────────────────────────────────────────────────┘
                         │
                         ▼
┌──────────────────────────────────────────────────────┐
│                 KIỂM TRA CHẤT LƯỢNG                  │
│  /review → /test → sửa lỗi → /review lại             │
└──────────────────────────────────────────────────────┘
                         │
                         ▼
┌──────────────────────────────────────────────────────┐
│                      TRIỂN KHAI                      │
│  /deploy (pre-launch checklist + security scan)      │
└──────────────────────────────────────────────────────┘
```

🧠 Mẹo Sử Dụng

1. Bắt đầu luôn bằng `/spec`

Dù project nhỏ hay lớn. 5 phút viết spec tiết kiệm 2 giờ sửa bug.

2. Đừng skip `/review`

Đây là lúc AI "tự soi" code của chính nó. Bạn sẽ bất ngờ vì số lỗi nó tự tìm ra.

3. Dùng `/code-simplify` khi code bắt đầu "nặng"

Sau 3-4 lần `/enhance`, code thường phình to. Chạy `/code-simplify` để dọn dẹp.

4. `/status` là "bản đồ" của bạn

Mở session mới? Chạy `/status` trước tiên để nhớ lại context.

5. `/orchestrate` = vũ khí hạng nặng

Chỉ dùng khi task **thực sự** cần 3+ agents. Tốn token nhưng cực mạnh.

6. Skill tự load — bạn KHÔNG cần gọi

Đừng lo về 38 skills. Hệ thống `intelligent-routing` sẽ tự chọn đúng skill cho đúng task.

7. Tag agent khi muốn chính xác

```
Bình thường:  /create (AI tự chọn agent)
Chính xác:    @mobile-developer /create (ép dùng mobile-developer)
```

8. Khi bị lỗi → `/debug` trước, KHÔNG tự sửa

```
❌ "Sửa lỗi này cho tôi" (AI sửa mò)
✅ /debug lỗi TypeError ở dòng 42 (AI điều tra có hệ thống)
```

9. Combo hay dùng nhất hàng ngày

```
/enhance → /preview → /review → /test
(Thêm feature → Xem → Kiểm tra → Test)
```

10. Combo làm sạch code

```
/code-simplify → /test → /review
(Dọn dẹp → Confirm không vỡ → Review lần cuối)
```